

Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Разработка кроссплатформенных приложений»

на тему «Обработка исключительных ситуаций»

Вариант №10

Выполнили: ст. гр. 23ВВИ1

Галкин А.П.

Казаков А.Д.

Приняли:

к.т.н. доцент Юрова О.В.

к.т.н. доцент Карамышева Н.С.

Пенза 2026

Цель работы: изучить механизм обработки исключительных ситуаций.

Задание:

Модифицировать приложение из предыдущей лабораторной работы, реализовав проверку вводимых данных с использованием механизма исключений. Необходимо создать свой класс, унаследованный от класса `Exception`, и генерировать исключение, если возникает попытка создать экземпляр класса `RecIntegral` со значениями, не являющимися числами в диапазоне от 0,000001 до 1000000. В качестве обработки исключения необходимо выводить диалог, содержащий предупреждение о некорректности введенных данных. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Ход работы:

- Создан пользовательский класс `InvalidRangeException`, унаследованный от `java.lang.Exception`, который хранит информацию о некорректном значении и передаёт сообщение об ошибке.
- В класс `RecIntegral` добавлены методы `validateRange()` и `validateParameters()`, вызываемые в конструкторе. При выходе параметров за допустимый диапазон или нарушении логики интегрирования генерируется исключение.
- В обработчиках кнопок `jButtonAddActionPerformed` и `jButtonSolveActionPerformed` добавлены блоки `try-catch`. При возникновении `InvalidRangeException`, `NumberFormatException` или `IllegalArgumentException` пользователю выводится модальное окно `JOptionPane` с пояснением ошибки.
- Графический интерфейс сохранён, добавлена только защита от некорректного ввода.

Листинг программы:

`InvalidRangeException.java`

```
package pkg23vvv1;
```

```
public class InvalidRangeException extends Exception {  
    double errVal;  
  
    public double getErrVal(){  
        return errVal;  
    }  
}
```

```

public InvalidRangeException(String message, double num){
    super(message);
    errVal = num;
}

public InvalidRangeException(String message){
    super(message);
}
}

RecIntegral.java
package pkg23vvv1;

public class RecIntegral {
    private static final double MIN_VALUE = 0.0000001;
    private static final double MAX_VALUE = 10000000;
    private double upperLim;
    private double lowLim;
    private double step;
    private double result;

    private void validateRange(double value, String fileName) throws InvalidRangeException {
        if (value < MIN_VALUE || value > MAX_VALUE){
            throw new InvalidRangeException(
                "Значение: "+ fileName + " должно быть в диапазоне от " +
                MIN_VALUE + " до " + MAX_VALUE + "\n", value
            );
        }
    }
}

```

```

private void validateParameters() throws InvalidRangeException {
    validateRange(lowLim, "Нижний предел");
    validateRange(upperLim, "Верхний предел");
    validateRange(step, "Шаг интегрирования");

    if (lowLim >= upperLim) {
        throw new InvalidRangeException("Ошибка: нижний предел должен быть меньше
верхнего предела");
    }
    if (step <= 0) {
        throw new InvalidRangeException("Ошибка: шаг интегрирования должен быть
положительным");
    }
    if (step > (upperLim - lowLim)) {
        throw new InvalidRangeException("Ошибка: шаг интегрирования не может быть
больше интервала интегрирования");
    }
}

```

```

public RecIntegral(double lowLim, double upperLim, double step) throws
InvalidRangeException{
    this.lowLim = lowLim;
    this.upperLim = upperLim;
    this.step = step;
    this.result = 0.0;
    validateParameters();
}

```

```

public RecIntegral(double lowLim, double upperLim, double step, double result) throws
InvalidRangeException{

```

```

    this.lowLim = lowLim;
    this.upperLim = upperLim;
    this.step = step;
    this.result = result;
    validateParameters();
}

```

```

public double GetUpperLim() {return upperLim;}
public double GetLowLim() {return lowLim;}
public double GetStep() {return step;}
public double GetResult() {return result;}
public void setResult(double result){this.result = result;}

```

```

public double calculate(double LowLim, double upperLim, double step){
    int n = (int)((upperLim - LowLim) / step);
    double integral = 0;
    for (int i = 1; i < n; i++) {
        double x = LowLim + i * step;
        if (Math.abs(x) > 1e-10) {
            integral += Math.exp(x) / x;
        }
    }
    double fa = (Math.abs(LowLim) > 1e-10) ? Math.exp(LowLim) / LowLim : 0;
    double fb = (Math.abs(upperLim) > 1e-10) ? Math.exp(upperLim) / upperLim : 0;
    double lastFullX = LowLim + n * step;
    double fLast = (Math.abs(lastFullX) > 1e-10) ? Math.exp(lastFullX) / lastFullX : 0;
    double lastStep = upperLim - lastFullX;

    if (Math.abs(lastStep) < 1e-10) {

```

```

        integral = (step / 2) * (fa + 2 * integral + fLast);
    } else {
        integral = (step / 2) * (fa + 2 * integral + fLast) + (lastStep / 2) * (fLast + fb);
    }
    return integral;
}
}

```

Скриншоты:

Формула: $(e^x)/x$

Нижний порог

Верхний порог

Шаг

Добавление из коллекции

Добавить

Очистить таблицу

Удалить

Вычислить

Нижний порог	Верхний порог	Шаг	Результат

Рисунок 1 - Окно приложения при запуске

Формула: $(e^x)/x$

Нижний порог:

Верхний порог:

Шаг:

Добавление из коллекции

Добавить

Очистить таблицу

Удалить

Вычислить

Нижний порог	Верхний порог	Шаг	Результат
1	5	0.5	38.782928156314796
0	5	0.5	2500041.3612193866

Рисунок 2 – Успешное добавление строки с корректными параметрами

Формула: $(e^x)/x$

Нижний порог:

Верхний порог:

Шаг:

Добавление из коллекции

Добавить

Очистить таблицу

Удалить

Вычислить

Нижний

Ошибка диапазона

Ошибка: нижний предел должен быть меньше верхнего предела

OK

56314796

12193866

ork/уник/ла

/work/уник

Рисунок 3 – Диалоговое окно при попытке ввести значение вне диапазона

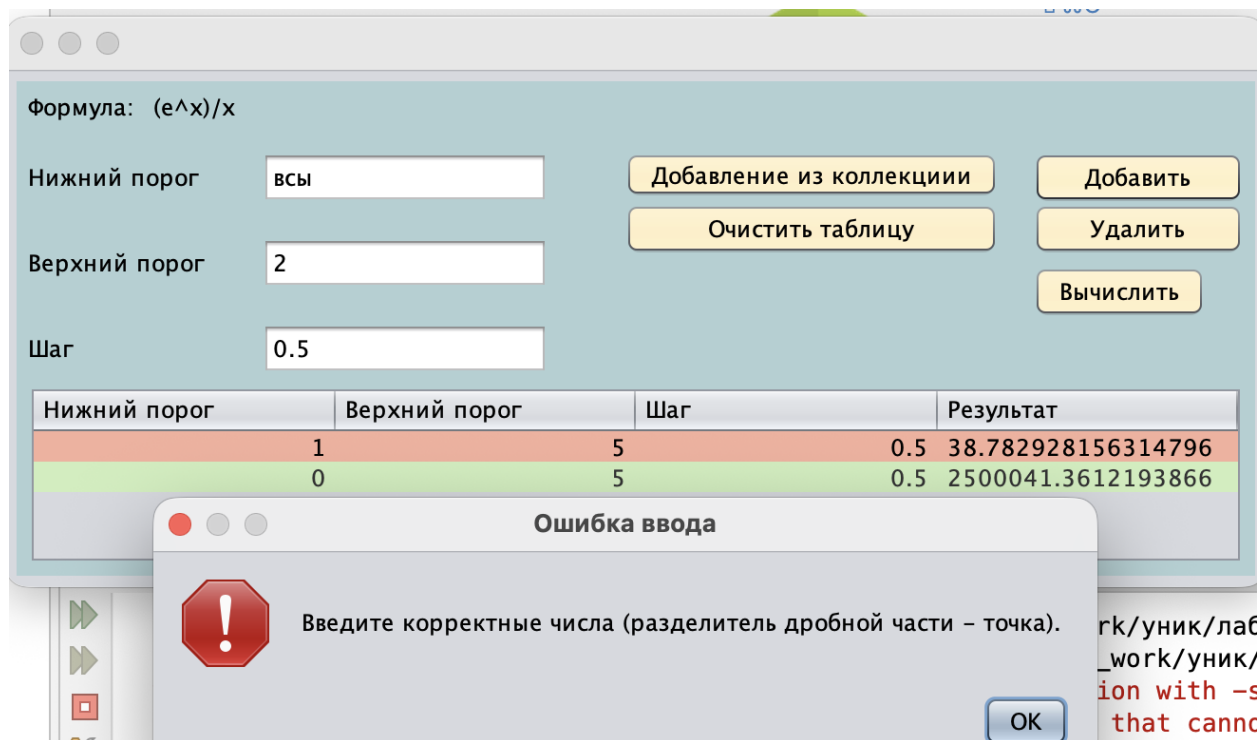


Рисунок 4 – Диалоговое окно при вводе нечисловых символов

Вывод:

В ходе выполнения лабораторной работы изучен механизм обработки исключительных ситуаций в Java. Реализован пользовательский класс `InvalidRangeException`, унаследованный от `Exception`, добавлена валидация входных данных в конструкторе класса `RecIntegral`, а также организована обработка исключений в графическом интерфейсе с выводом информативных диалоговых окон. Приложение стало устойчивым к некорректному вводу пользователя и логическим ошибкам параметров интегрирования.